

LAPORAN PRAKTIKUM STRUKTUR DATA

“Implementasi Struktur Data Single Linked List pada Java”

disusun Oleh:

Muhammad Faiz An Anri

NIM 2511532021

Dosen Pengampu: Dr. Wahyudi S.T M.T

Asisten Pratikum: Jovantri Immanuel Gulo



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

DAFTAR ISI

DAFTAR ISI	i
KATA PENGANTAR.....	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum.....	1
BAB II PEMBAHASAN	2
2.1 Program NodeSSL	2
2.2 Program TambahSSL.....	2
2.3 Program PencarianSSL	5
2.4 Program HapusSSL	6
BAB III TUGAS	10
3.1 Program AntrianLoket.....	10
3.2 Program RumahSakit	11
BAB IV KESIMPULAN	15
4.1 Ringkasan Hasil Praktikum	15
DAFTAR PUSTAKA.....	iii

KATA PENGANTAR

Ucapan syukur kita lafalkan kepada Allah SWT atas limpahan rahmat dan karunia-Nya sehingga laporan praktikum ini dapat disusun dan diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk pelaksanaan kegiatan praktikum pada mata kuliah Struktur Data.

Ucapan terima kasih disampaikan kepada Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu, serta kepada asisten laboratorium yang telah memberikan bimbingan selama kegiatan praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi penulisan maupun isi. Oleh karena itu, saran dan kritik yang bersifat membangun sangat diharapkan demi perbaikan di masa yang akan datang. Harapannya, laporan ini dapat memberikan manfaat dan menjadi referensi bagi pembelajaran konsep perulangan dalam bahasa pemrograman Java.

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Salah satu kelemahan utama Array adalah ukuran memorinya yang tetap (statis), sehingga sulit menyesuaikan kapasitas saat data bertambah atau berkurang. Untuk mengatasi keterbatasan tersebut, digunakan struktur data Linked List atau Senarai Berantai. Tidak seperti Array yang menyimpan elemen pada lokasi memori yang berurutan, Linked List menggunakan alokasi memori secara dinamis melalui elemen yang disebut Node (Simpul).

Pada Single Linked List (SLL), setiap Node memiliki dua komponen utama, yaitu data yang disimpan dan pointer (penunjuk) yang berisi alamat memori menuju Node selanjutnya.

1.2 Tujuan Praktikum

Tujuan dari praktikum ini adalah:

1. Memahami konsep dasar Node pada struktur data Single Linked List.
2. Mengimplementasikan operasi penambahan data (Insertion) di berbagai posisi, yaitu di awal (front), di akhir (end), maupun pada posisi spesifik (insertPos).
3. Mengimplementasikan operasi penelusuran (Traversal), pencarian data (Search), serta penghapusan data (Deletion) tanpa memutus rantai pointer memori.

BAB 2

PEMBAHASAN

3.1 Program NodeSLL

Class NodeSLL digunakan sebagai cetak biru (blueprint) bagi setiap elemen pembentuk Linked List. Class ini memiliki dua atribut utama, yaitu data bertipe integer yang digunakan untuk menyimpan nilai, serta next yang bertipe objek Node untuk menyimpan referensi menuju simpul berikutnya. Proses inisialisasi dilakukan melalui konstruktor, dengan nilai yang dimasukkan disimpan ke dalam atribut data, sementara pointer next secara default bernilai null.

```
1 package pekan5_2511532021;
2
3 public class NodeSSL_2511532021 {
4     int data_2021;
5
6     NodeSSL_2511532021 next_2021;
7
8     public NodeSSL_2511532021(int data_2021) {
9         this.data_2021 = data_2021;
10        this.next_2021 = null;
11    }
12 }
13
```

3.2 Program TambahSLL

Program Java tersebut mengimplementasikan operasi penambahan node pada Single Linked List (SLL). Class TambahSSL_2511532021 memiliki method untuk menambah node di depan (insertAtFront_2021()), di belakang (insertAtEnd_2021()), dan pada posisi tertentu (insertPos_2021()). Selain itu, terdapat method printList_2021() untuk menampilkan isi linked list. Pada method main, program membuat linked list awal lalu melakukan beberapa penambahan node untuk menunjukkan cara kerja manipulasi data secara dinamis pada linked list.

```

1 package pekan5;
2
3 public class TambahSSL_2511532021 {
4     public static NodeSSL_2511532021 insertAtFront_2021(NodeSSL_2511532021 head_2021, int value_2021) {
5         NodeSSL_2511532021 new_node_2021 = new NodeSSL_2511532021(value_2021);
6         new_node_2021.next_2021 = head_2021;
7         return new_node_2021;
8     }
9
10    // adds node in the end
11    public static NodeSSL_2511532021 insertAtEnd_2021(NodeSSL_2511532021 head_2021, int value_2021) {
12        // create node with a value
13        NodeSSL_2511532021 newNode_2021 = new NodeSSL_2511532021(value_2021);
14
15        // if list is empty, then node becomes head
16        if (head_2021 == null) {
17            return newNode_2021;
18        }
19
20        // store head to last
21        NodeSSL_2511532021 last_2021 = head_2021;
22        // find to the last node
23        while (last_2021.next_2021 != null) {
24            last_2021 = last_2021.next_2021;
25        }
26
27        // change the pointer
28        last_2021.next_2021 = newNode_2021;
29        return head_2021;
30    }
31
32    static NodeSSL_2511532021 getNode_2021(int data_2021) {
33        return new NodeSSL_2511532021(data_2021);
34    }
35
36    static NodeSSL_2511532021 insertPos_2021(NodeSSL_2511532021 headNode_2021, int pos_2021, int val_2021) {
37        NodeSSL_2511532021 head_2021 = headNode_2021;
38        if (pos_2021 < 1) {
39            System.out.println("Invalid position.");
40        }
41        if (pos_2021 == 1) {
42            NodeSSL_2511532021 new_node_2021 = new NodeSSL_2511532021(val_2021);
43            new_node_2021.next_2021 = head_2021;
44            return new_node_2021;
45        } else {
46            while (pos_2021-- != 0) {
47                if (pos_2021 == 1) {
48                    NodeSSL_2511532021 newNode_2021 = getNode_2021(val_2021);
49                    newNode_2021.next_2021 = headNode_2021.next_2021;
50                    headNode_2021.next_2021 = newNode_2021;
51                    break;
52                }
53                headNode_2021 = headNode_2021.next_2021;
54            }
55            if (pos_2021 != 1)
56                System.out.println("Posisi di luar jangkauan");

```

```

55            if (pos_2021 != 1)
56                System.out.println("Posisi di luar jangkauan");
57        }
58        return head_2021;
59    }
60
61    public static void printList_2021(NodeSSL_2511532021 head_2021) {
62        NodeSSL_2511532021 curr_2021 = head_2021;
63        while (curr_2021.next_2021 != null) {
64            System.out.print(curr_2021.data_2021 + "-->");
65            curr_2021 = curr_2021.next_2021;
66        }
67        if (curr_2021.next_2021 == null) {
68            System.out.print(curr_2021.data_2021);
69            System.out.println();
70        }
71    }
72
73    public static void main(String[] args) {
74        // example linked list 2 -> 3 -> 5 -> 6
75        NodeSSL_2511532021 head_2021 = new NodeSSL_2511532021(2);
76        head_2021.next_2021 = new NodeSSL_2511532021(3);
77        head_2021.next_2021.next_2021 = new NodeSSL_2511532021(5);
78        head_2021.next_2021.next_2021.next_2021 = new NodeSSL_2511532021(6);
79
80        // print original list
81        System.out.print("Senarai berantai awal : ");
82        printList_2021(head_2021);
83
84        // adds new node to the head
85        System.out.print("tambah 1 simpul di depan: ");
86        int data_2021 = 1;
87        head_2021 = insertAtFront_2021(head_2021, data_2021);
88
89        // print updated list
90        printList_2021(head_2021);
91
92        // adds new node to the end
93        System.out.print("tambah 1 simpul di belakang: ");
94        int data2_2021 = 7;
95        head_2021 = insertAtEnd_2021(head_2021, data2_2021);
96
97        // print updated list
98        printList_2021(head_2021);
99
100        System.out.print("tambah 1 simpul ke data 4: ");
101        int data3_2021 = 4;
102        int pos_2021 = 4;
103        head_2021 = insertPos_2021(head_2021, pos_2021, data3_2021);
104
105        // print updated list
106        printList_2021(head_2021);
107    }

```

3.3 Program PencarianSLL

Karena Linked List tidak memiliki indeks seperti Array, proses pencarian data dilakukan secara berurutan. Program PencarianSLL menerapkan algoritma pencarian linier melalui method searchKey dan traversal. Pencarian dilakukan menggunakan perulangan while dengan bantuan variabel curr yang bergerak dari head hingga mencapai null. Pada setiap langkah, data pada node dibandingkan dengan nilai kunci (key) sampai data ditemukan atau seluruh node selesai ditelusuri.

```

1 package pekan5_2511532021;
2
3 public class PencarianSLL_2511532021 {
4     static boolean searchKey_2021(NodeSSL_2511532021 head_2021, int key_2021) {
5         NodeSSL_2511532021 curr_2021 = head_2021;
6         while(curr_2021 != null) {
7             if (curr_2021.data_2021 == key_2021) {
8                 return true;
9             }
10            curr_2021 = curr_2021.next_2021;
11        }
12        return false;
13    }
14
15
16    public static void traversal_2021(NodeSSL_2511532021 head_2021) {
17        NodeSSL_2511532021 curr_2021 = head_2021;
18
19        while(curr_2021 != null) {
20            System.out.print(" " + curr_2021.data_2021);
21            curr_2021 = curr_2021.next_2021;
22        }
23        System.out.println("");
24    }
25
26    public static void main(String[] args) {
27        NodeSSL_2511532021 head_2021 = new NodeSSL_2511532021(14);
28        head_2021.next_2021 = new NodeSSL_2511532021(21);
29        head_2021.next_2021.next_2021 = new NodeSSL_2511532021(13);
30        head_2021.next_2021.next_2021.next_2021 = new NodeSSL_2511532021(30);
31        head_2021.next_2021.next_2021.next_2021.next_2021 = new NodeSSL_2511532021(10);
32        System.out.print("Penelusuran SLL : ");
33        traversal_2021(head_2021);
34
35        // data that will be searched
36        int key_1009 = 30;
37        System.out.print("cari data " + key_1009 + " = ");
38        if (searchKey_2021(head_2021, key_1009))
39            System.out.println("data ditemukan");
40        else
41            System.out.println("data tidak ditemukan");
42    }
43 }
44 }
45
46

```

3.4 Program HapusSLL

Program HapusSLL berisi operasi penghapusan node pada Single Linked List yang harus dilakukan dengan hati-hati agar struktur rantai tetap terhubung. Method `deleteHead` menghapus node pertama dengan memindahkan head ke node berikutnya (`head.next`). Method `removeLastNode` menelusuri linked list hingga menemukan node sebelum terakhir, kemudian mengubah pointer `next` menjadi `null`. Sementara itu, method `deleteNode` menggunakan dua pointer, yaitu `prev` dan `temp`, untuk menemukan node target lalu menghubungkan `prev.next` langsung ke node setelah `temp`.

```

1 package pekan5_2511532021;
2
3 public class HapusSLL_2511532021 {
4
5     // function to delete head node
6     public static NodeSSL_2511532021 deleteHead_2021(NodeSSL_2511532021 head_2021) {
7         if (head_2021 == null)
8             return null;
9
10        head_2021 = head_2021.next_2021;
11        return head_2021;
12    }
13
14    // function to delete last node
15    public static NodeSSL_2511532021 removeLastNode(NodeSSL_2511532021 head_2021) {
16        if (head_2021 == null) {
17            return null;
18        }
19
20        if (head_2021.next_2021 == null) {
21            return null;
22        }
23
24        // search second last node
25        NodeSSL_2511532021 secondLast_2021 = head_2021;
26        while (secondLast_2021.next_2021.next_2021 != null) {
27            secondLast_2021 = secondLast_2021.next_2021;
28        }
29
30        secondLast_2021.next_2021 = null;
31        return head_2021;
32    }
33
34    // function to delete node in a certain position
35    public static NodeSSL_2511532021 deleteNode_2021(NodeSSL_2511532021 head_2021, int pos_2021) {
36        NodeSSL_2511532021 temp_2021 = head_2021;
37        NodeSSL_2511532021 prev_2021 = null;
38
39        if (temp_2021 == null)
40            return head_2021;
41
42        if (pos_2021 == 1) {
43            head_2021 = temp_2021.next_2021;
44            return head_2021;
45        }
46
47        for (int i_2021 = 1; temp_2021 != null && i_2021 < pos_2021; i_2021++) {
48            prev_2021 = temp_2021;
49            temp_2021 = temp_2021.next_2021;
50        }
51
52        if (temp_2021 != null) {
53            prev_2021.next_2021 = temp_2021.next_2021;
54        } else {
55            System.out.println("Data tidak ditemukan");
56        }
57        return head_2021;
58    }
59
60    // function to print SLL
61    public static void printList_2021(NodeSSL_2511532021 head_2021) {
62        NodeSSL_2511532021 curr_2021 = head_2021;
63        while (curr_2021.next_2021 != null) {
64            System.out.print(curr_2021.data_2021 + "-->");
65            curr_2021 = curr_2021.next_2021;
66        }
67        if (curr_2021.next_2021 == null) {
68            System.out.print(curr_2021.data_2021);
69        }
70        System.out.println();
71    }
72
73 }

```


BAB 3

TUGAS

3.5 Program Pasien

Class Pasien_2511532021 merupakan representasi objek node pada Single Linked List yang digunakan untuk menyimpan data pasien. Class ini memiliki atribut nama_2021, keluhan_2021, dan noAntrian_2021 untuk menyimpan informasi pasien, serta atribut next_2021 yang berfungsi sebagai pointer menuju node pasien berikutnya. Inisialisasi data dilakukan melalui konstruktor, sedangkan method getter dan setter digunakan untuk mengakses serta mengubah nilai atribut secara terkontrol.

```
1 package pekan5_2511532021;
2
3 public class Pasien_2511532021 {
4
5     private String nama_2021;
6     private String keluhan_2021;
7     private int noAntrian_2021;
8     private Pasien_2511532021 next_2021;
9
10    public Pasien_2511532021(String nama_2021, String keluhan_2021, int noAntrian_2021) {
11        this.nama_2021 = nama_2021;
12        this.keluhan_2021 = keluhan_2021;
13        this.noAntrian_2021 = noAntrian_2021;
14        this.next_2021 = null;
15    }
16
17    public String getNama_2021() { return nama_2021; }
18    public void setNama_2021(String nama_2021) { this.nama_2021 = nama_2021; }
19
20    public String getKeluhan_2021() { return keluhan_2021; }
21    public void setKeluhan_2021(String keluhan_2021) { this.keluhan_2021 = keluhan_2021; }
22
23    public int getNoAntrian_2021() { return noAntrian_2021; }
24    public void setNoAntrian_2021(int noAntrian_2021) { this.noAntrian_2021 = noAntrian_2021; }
25
26    public Pasien_2511532021 getNext_2021() { return next_2021; }
27    public void setNext_2021(Pasien_2511532021 next_2021) { this.next_2021 = next_2021; }
28 }
```

3.2 Program RumahSakit

Program RumahSakit_2511532021 merupakan implementasi Single Linked List (SLL) untuk mensimulasikan sistem antrian pasien di rumah sakit. Data pasien disimpan menggunakan objek Pasien_2511532021 yang berisi nama pasien, keluhan, nomor antrian, serta pointer menuju pasien berikutnya. Variabel head_2021 berfungsi sebagai penanda awal linked list, sedangkan counterAntrian_2021 digunakan untuk memberikan nomor antrian otomatis dan totalPasien_2021 untuk menghitung jumlah pasien yang masih berada dalam antrian.

Method daftarkanPasien_2021() digunakan untuk menambahkan pasien baru ke bagian akhir antrian dengan cara menelusuri linked list hingga node terakhir, lalu menghubungkannya dengan node baru. Method panggilPasien_2021() menerapkan konsep delete head, yaitu menghapus pasien terdepan dengan memindahkan head ke node berikutnya. Selain itu, terdapat method tampilkanAntrian_2021() untuk menampilkan seluruh data pasien secara berurutan, cariPasien_2021() untuk melakukan pencarian pasien berdasarkan nama menggunakan traversal linear, serta cekStatusAntrian_2021() untuk menampilkan jumlah pasien dan pasien yang berada di posisi terdepan.

Pada method main, program dijalankan secara interaktif menggunakan menu berbasis Scanner. Pengguna dapat memilih berbagai operasi seperti menambah pasien, memanggil pasien, menampilkan antrian, mencari pasien, dan mengecek status antrian. Program akan terus berjalan dalam perulangan do-while hingga pengguna memilih menu keluar, sehingga simulasi antrian rumah sakit dapat dilakukan secara dinamis menggunakan struktur data linked list.

```

1 package peka5_2511532021;
2 import java.util.Scanner;
3
4 public class RumahSakit_2511532021 {
5     private Pasien_2511532021 head_2021;
6     private int counterAntrian_2021;
7     private int totalPasien_2021;
8
9     public RumahSakit_2511532021() {
10         this.head_2021 = null;
11         this.counterAntrian_2021 = 0;
12         this.totalPasien_2021 = 0;
13     }
14
15     public void daftarkanPasien_2021(String nama_2021, String keluhan_2021) {
16         counterAntrian_2021++;
17         Pasien_2511532021 nodeBaru_2021 = new Pasien_2511532021(nama_2021, keluhan_2021, counterAntrian_2021);
18
19         if (head_2021 == null) {
20             head_2021 = nodeBaru_2021;
21         } else {
22             Pasien_2511532021 temp_2021 = head_2021;
23             while (temp_2021.getNext_2021() != null) {
24                 temp_2021 = temp_2021.getNext_2021();
25             }
26             temp_2021.setNext_2021(nodeBaru_2021);
27         }
28         totalPasien_2021++;
29         System.out.println("Pasien berhasil didaftarkan! Nomor Antrian: " + counterAntrian_2021);
30     }
31
32     public void panggilPasien_2021() {
33         if (head_2021 == null) {
34             System.out.println("Antrian kosong. Tidak ada pasien yang dapat dipanggil.");
35             return;
36         }
37         System.out.println("Memanggil Pasien.");
38         System.out.println("No Antrian: " + head_2021.getNoAntrian_2021() + " | Nama: " + head_2021.getNama_2021() + " | Keluhan: " + head_2021.getKeluhan_2021());
39
40         head_2021 = head_2021.getNext_2021();
41         totalPasien_2021--;
42     }
43
44     public void tampilkanAntrian_2021() {}
45
46     if (head_2021 == null) {
47         System.out.println("Antrian kosong.");
48         return;
49     }
50     System.out.println("=== Daftar Antrian Pasien ===");
51     Pasien_2511532021 temp_2021 = head_2021;
52     int posisi_2021 = 1;
53     while (temp_2021 != null) {
54         System.out.println(posisi_2021 + ". [No." + temp_2021.getNoAntrian_2021() + "] " + temp_2021.getNama_2021() + " - " + temp_2021.getKeluhan_2021());
55         temp_2021 = temp_2021.getNext_2021();
56         posisi_2021++;
57     }
58 }
59
60 public void cariPasien_2021(String cariNama_2021) {
61     if (head_2021 == null) {
62         System.out.println("Antrian kosong.");
63         return;
64     }
65     Pasien_2511532021 temp_2021 = head_2021;
66     boolean ditemukan_2021 = false;
67     int posisi_2021 = 1;
68
69     while (temp_2021 != null) {
70         if (temp_2021.getNama_2021().equalsIgnoreCase(cariNama_2021)) {
71             System.out.println("Pasien ditemukan pada antrian ke-" + posisi_2021 + ":");
72             System.out.println("No Antrian: " + temp_2021.getNoAntrian_2021() + " | Nama: " + temp_2021.getNama_2021() + " | Keluhan: " + temp_2021.getKeluhan_2021());
73             ditemukan_2021 = true;
74         }
75         temp_2021 = temp_2021.getNext_2021();
76         posisi_2021++;
77     }
78 }

```

```

        temp_2021 = temp_2021.getNext_2021();
        posisi_2021++;
    }
    if (!ditemukan_2021) {
        System.out.println("Pasien dengan nama '" + cariNama_2021 + "' tidak ditemukan dalam antrian.");
    }
}

public void cekStatusAntrian_2021() {
    if (head_2021 == null) {
        System.out.println("Antrian saat ini kosong.");
        return;
    }
    System.out.println("=== Status Antrian ===");
    System.out.println("Total pasien dalam antrian: " + totalPasien_2021);
    System.out.println("Pasien selanjutnya (Terdepan): [No." + head_2021.getNoAntrian_2021() + "] " + head_2021.getNama_2021());
}

public static void main(String[] args) {
    Scanner sc_2021 = new Scanner(System.in);
    RumahSakit_2511532021 rs_2021 = new RumahSakit_2511532021();
    int pilihan_2021;

    do {
        System.out.println("\nAntrian Rumah Sakit NDM: 2511532021");
        System.out.println("1. Daftarkan Pasien (Insert)");
        System.out.println("2. Panggil Pasien (Delete Head)");
        System.out.println("3. Tampilkan Antrian (Display)");
        System.out.println("4. Cari Pasien (Search)");
        System.out.println("5. Cek Status Antrian");
        System.out.println("6. Keluar");
        System.out.print("Pilihan: ");
        pilihan_2021 = sc_2021.nextInt();
        sc_2021.nextLine();

        switch (pilihan_2021) {
            case 1:
                System.out.print("Masukkan Nama Pasien: ");
                String nama_2021 = sc_2021.nextLine();
                System.out.print("Masukkan keluhan: ");
                String keluhan_2021 = sc_2021.nextLine();
                rs_2021.daftarkanPasien_2021(nama_2021, keluhan_2021);
                break;
            case 2:
                rs_2021.panggilPasien_2021();
                break;
            case 3:
                rs_2021.tampilkanAntrian_2021();
                break;
            case 4:
                System.out.print("Masukkan Nama Pasien yang dicari: ");
                String cari_2021 = sc_2021.nextLine();
                rs_2021.cariPasien_2021(cari_2021);
                break;
            case 5:
                rs_2021.cekStatusAntrian_2021();
                break;
            case 6:
                System.out.println("Keluar dari program simulasi Rumah Sakit.");
                break;
            default:
                System.out.println("Pilihan tidak valid.");
        }
    } while (pilihan_2021 != 6);
    sc_2021.close();
}

```

BAB 4

KESIMPULAN

4.1 Ringkasan Hasil Praktikum

Berdasarkan praktikum yang telah dilakukan, dapat dipahami bahwa Single Linked List memiliki kelebihan dalam pengelolaan memori secara dinamis dibandingkan Array, karena memori hanya dialokasikan ketika node baru ditambahkan. Setiap operasi pada struktur data ini, seperti penyisipan, pencarian, dan penghapusan data, bergantung pada pengaturan referensi atau pointer (next) antar node. Linked List juga lebih efektif untuk melakukan perubahan data pada bagian awal list (head), namun memiliki kekurangan pada proses akses data karena tidak menggunakan indeks tetap, sehingga penelusuran harus dilakukan secara berurutan dari node pertama.